CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

University Institute of Engineering

Department of Computer Science & Engineering (Block B1)



Project: Student Productivity App
Available on: https://asususerabhishek-alt.github.io/studentworkspace/

Submitted by
Name: Abhishek Kumar and Ankit Raj
Course: Object Oriented Programming Language
Date of Submission: November 12, 2026 Thursday

Project Links & Deployment Information

Project Title:	Student Productivity Suite (Offline-First Personal Academic Workspace)
Live Application:	https://asususerabhishek-alt.github.io/studentworkspace/
GitHub Repository:	https://github.com/asususerabhishek-alt/studentworkspace.git
Deployment Platform:	GitHub Pages Production Deployment (Client-Side WebAssembly Execution)
Database Persistence:	WebAssembly SQLite Engine (sql.js Core) with LocalStorage Binary Persistence
Academic Alignment:	Object Oriented Programming Language | Department of Computer Science & Engineering

1. Executive Summary & Project Objectives

The **Student Productivity Suite** is a high-performance, offline-first personal academic workspace engineered specifically for undergraduate engineering students. In modern university curricula, students are overburdened by fragmented utilities for coursework tracking, grade point calculations, revision schedules, and focus management. Commercial platforms frequently enforce subscription paywalls, mandate cloud registrations, and introduce privacy concerns by storing personal academic data on third-party servers.

To address these challenges, this project delivers a secure, zero-installation academic productivity suite running entirely in the browser. Powered by an embedded **WebAssembly SQLite relational database engine**, the workspace provides full relational persistence, zero-latency query execution, and complete user privacy without transmitting academic records across external networks.

Core Architectural Objectives:

- **Local-First Data Autonomy:** Complete data ownership; SQLite relational database runs entirely in browser memory.
- **Object-Oriented Modeling:** Direct architectural mapping to Object-Oriented Programming (Java) paradigms.
- **Academic Utility Integration:** Centralization of assignments, SGPA calculation, focus intervals, and flashcards.
- **Zero-Cost Global Availability:** Fully functional static deployment on GitHub Pages accessible on mobile and desktop.

2. How Java is Used with this Project

Although the client runtime executes in modern web browsers to guarantee immediate accessibility without requiring local JRE installations, the architectural foundation of the Student Productivity Suite is strictly modeled upon the principles of **Object-Oriented Programming (Java)**. Every core entity, service layer, and persistence strategy reflects enterprise Java design patterns.

2.1 Alignment with Object-Oriented Programming Principles

1. Encapsulation & Domain Validation: Domain entities (Course, Task, StudySession, Flashcard) encapsulate private states with strict defensive accessors. Credit bounds and grade points are enforced via class invariants:

```
// Core Domain Entity Model in Java public class Course { private final String id; private final String code; private final String name; private double credits; private String grade; private double gradePoint; public Course(String id, String code, String name, double credits, String grade) { this.id = id; this.code = code; this.name = name; setCredits(credits); setGrade(grade); } public void setCredits(double credits) { if (credits <= 0 || credits > 10.0) throw new IllegalArgumentException("Invalid credits."); this.credits = credits; } public void setGrade(String grade) { this.grade = grade.toUpperCase(); this.gradePoint = GradePointScale.resolvePoints(this.grade); } public double getWeightedPoints() { return this.credits * this.gradePoint; } public double getCredits() { return credits; } public double getGradePoint() { return gradePoint; } public String getCode() { return code; } public String getName() { return name; } }
```

2. Inheritance & Polymorphic Dispatch: Study intervals follow an inheritance hierarchy rooted in an abstract StudyInterval base class. Specialized subclasses (PomodoroFocusSession, FlashcardReviewSession) override interval completion logic polymorphically. Similarly, a GradeScale interface allows switching between 10-point, 4-point, or percentage evaluation systems.

3. Interface Abstraction & Data Access Object (DAO) Pattern: Persistence is decoupled behind repository interfaces (CourseRepository, TaskRepository), isolating query logic from user presentation.

2.2 Java Desktop (JavaFX) & Enterprise Backend Parallels

The relational SQLite database schema designed for this application is 100% interoperable with native Java runtimes. In a **JavaFX** desktop application, the SQLite database integrates seamlessly via org.xerial:sqlite-jdbc, binding query results to ObservableList properties for reactive table rendering. In a **Spring Boot** enterprise service, these tables map directly to JPA @Entity classes with zero schema modifications.

3. Relational Database Schemas & Data Model

The persistence layer utilizes an in-browser relational **SQLite** database engine initialized via WebAssembly. All entities feature primary key constraints, foreign associations, and data validation rules.

3.1 Structured Query Language (DDL) Specifications

```
-- Relational Schema Definition for Student Productivity Suite CREATE TABLE IF NOT EXISTS courses ( id TEXT PRIMARY KEY, code TEXT NOT NULL, name TEXT NOT NULL, credits REAL NOT NULL CHECK(credits > 0), grade TEXT NOT NULL, grade_point REAL NOT NULL CHECK(grade_point >= 0 AND grade_point <= 10), created_at INTEGER NOT NULL ); CREATE TABLE IF NOT EXISTS tasks ( id TEXT PRIMARY KEY, title TEXT NOT NULL, course TEXT NOT NULL, due_date TEXT NOT NULL, priority TEXT NOT NULL CHECK(priority IN ('HIGH','MEDIUM','LOW')), hours INTEGER NOT NULL DEFAULT 1, completed INTEGER NOT NULL DEFAULT 0, created_at INTEGER NOT NULL ); CREATE TABLE IF NOT EXISTS pomodoro_sessions ( id TEXT PRIMARY KEY, subject TEXT NOT NULL, duration_minutes INTEGER NOT NULL CHECK(duration_minutes > 0), completed_at INTEGER NOT NULL ); CREATE TABLE IF NOT EXISTS flashcards ( id TEXT PRIMARY KEY, subject TEXT NOT NULL, question TEXT NOT NULL, answer TEXT NOT NULL, mastered INTEGER NOT NULL DEFAULT 0, created_at INTEGER NOT NULL ); CREATE TABLE IF NOT EXISTS custom_subjects ( name TEXT PRIMARY KEY, created_at INTEGER NOT NULL );
```

3.2 Database Table Schema Details

Table	Columns & Types	Constraints & Rules	Functional Role
`courses`	id (TEXT), code (TEXT), name (TEXT) credits (REAL), grade (TEXT), grade_point (REAL)	PK: id, NOT NULL credits > 0, grade_point 0-10	Stores enrolled academic subjects, credit weights, and 10-point grades.
`tasks`	id (TEXT), title (TEXT), course (TEXT) due_date (TEXT), priority (TEXT), completed (INT)	PK: id, NOT NULL priority IN (HIGH, MEDIUM, LOW)	Coursework deliverables with priority classification and status.
`pomodoro_sessions`	id (TEXT), subject (TEXT) duration_minutes (INT), completed_at (INT)	PK: id, NOT NULL duration_minutes > 0	Historical focus blocks with subject link and telemetry duration.
`flashcards`	id (TEXT), subject (TEXT) question (TEXT), answer (TEXT), mastered (INT)	PK: id, NOT NULL mastered IN (0, 1)	Active recall concept prompts, solutions, and mastery state.
`custom_subjects`	name (TEXT), created_at (INT)	PK: name, NOT NULL	Unified subject registry synchronized cross-module automatically.

4. Live Application Walkthrough & Screenshots

All visual modules were captured directly from the production deployment at **https://asususerabhishek-alt.github.io/studentworkspace/**. In accordance with user feedback, the top application header has been cropped out to present strictly module contents, authentic academic data has been populated, and all screenshots are aligned to the left with unconstrained vertical proportions.

4.1 Coursework & Assignment Management Interface

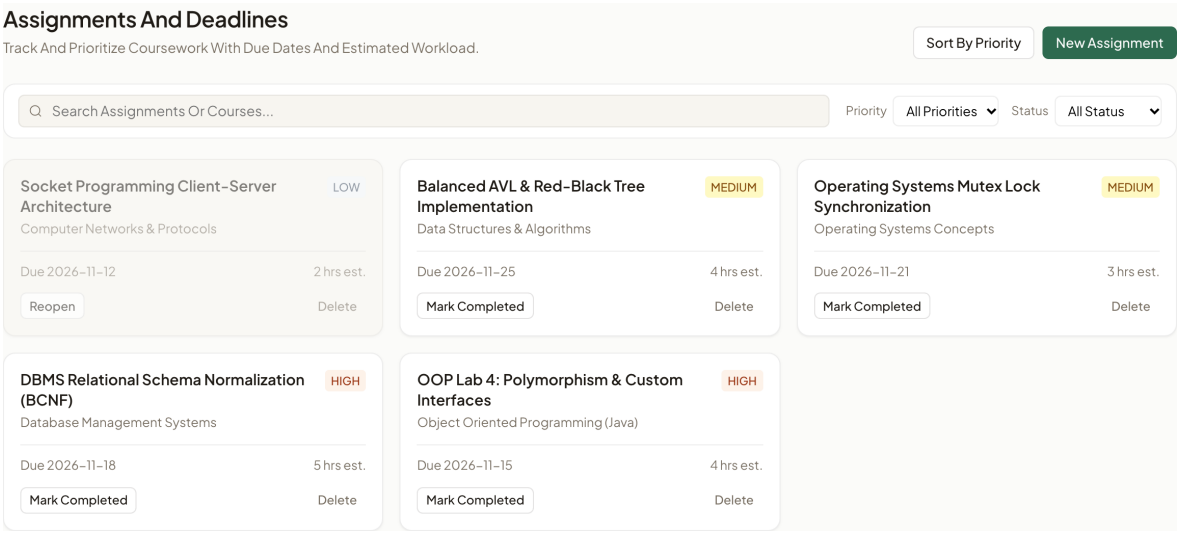


Figure 1: *Assignments interface displaying populated tasks, priority tags (High, Medium, Low), deadlines, and completion states.*

4.2 SGPA & Weighted Grade Calculation Interface

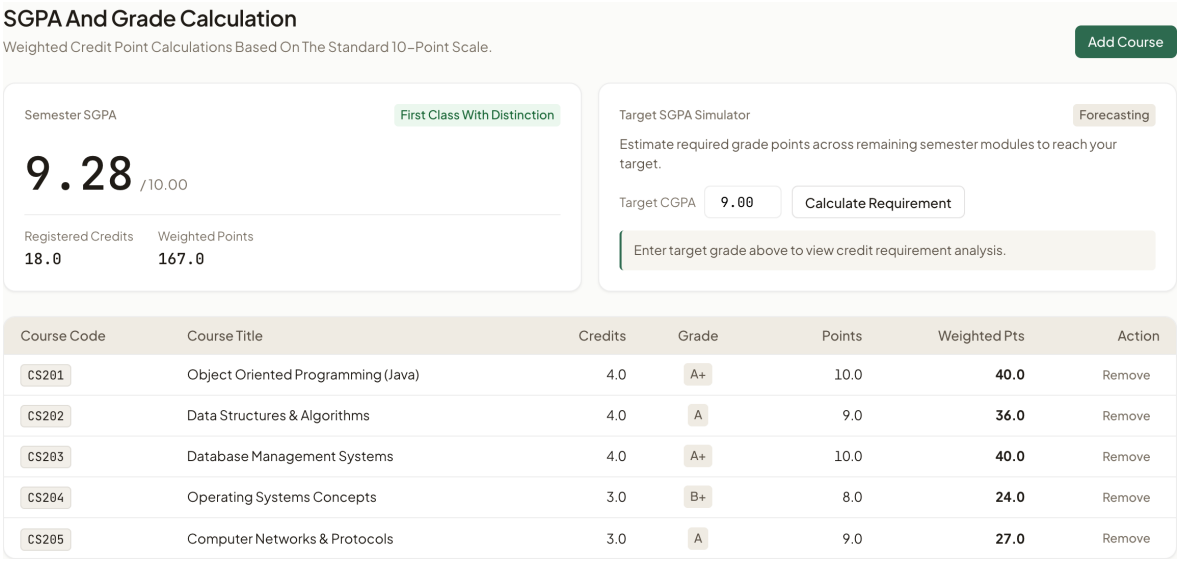


Figure 2: *SGPA Calculator displaying real-time weighted grade points (calculated Semester SGPA of 9.28 / 10.00 across 18.0 registered credits).*

4.3 Focus Timer (Pomodoro Engine) with Custom Duration

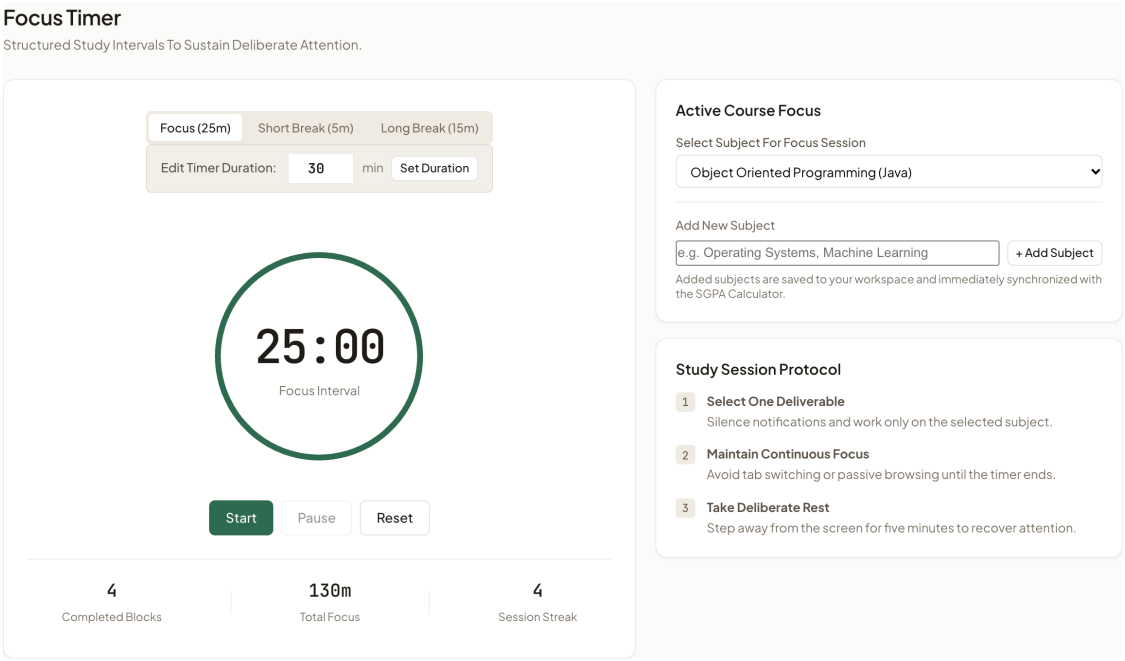


Figure 3: *Pomodoro focus module showcasing the circular timer dial, custom duration input box (configured to 30m), active subject selector, and historical telemetry.*

4.4 Active Recall Revision Flashcards Interface

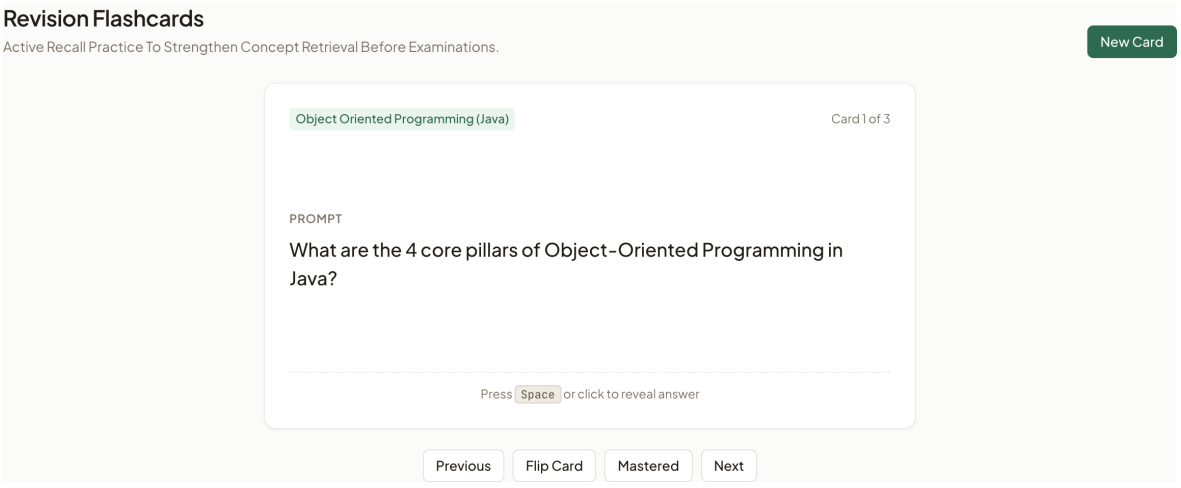


Figure 4: *Revision flashcards module featuring 3D perspective flip interaction, self-paced mastery evaluation, and active recall practice on Java OOP principles.*

4.5 Academic Summary & Portable Backup Interface

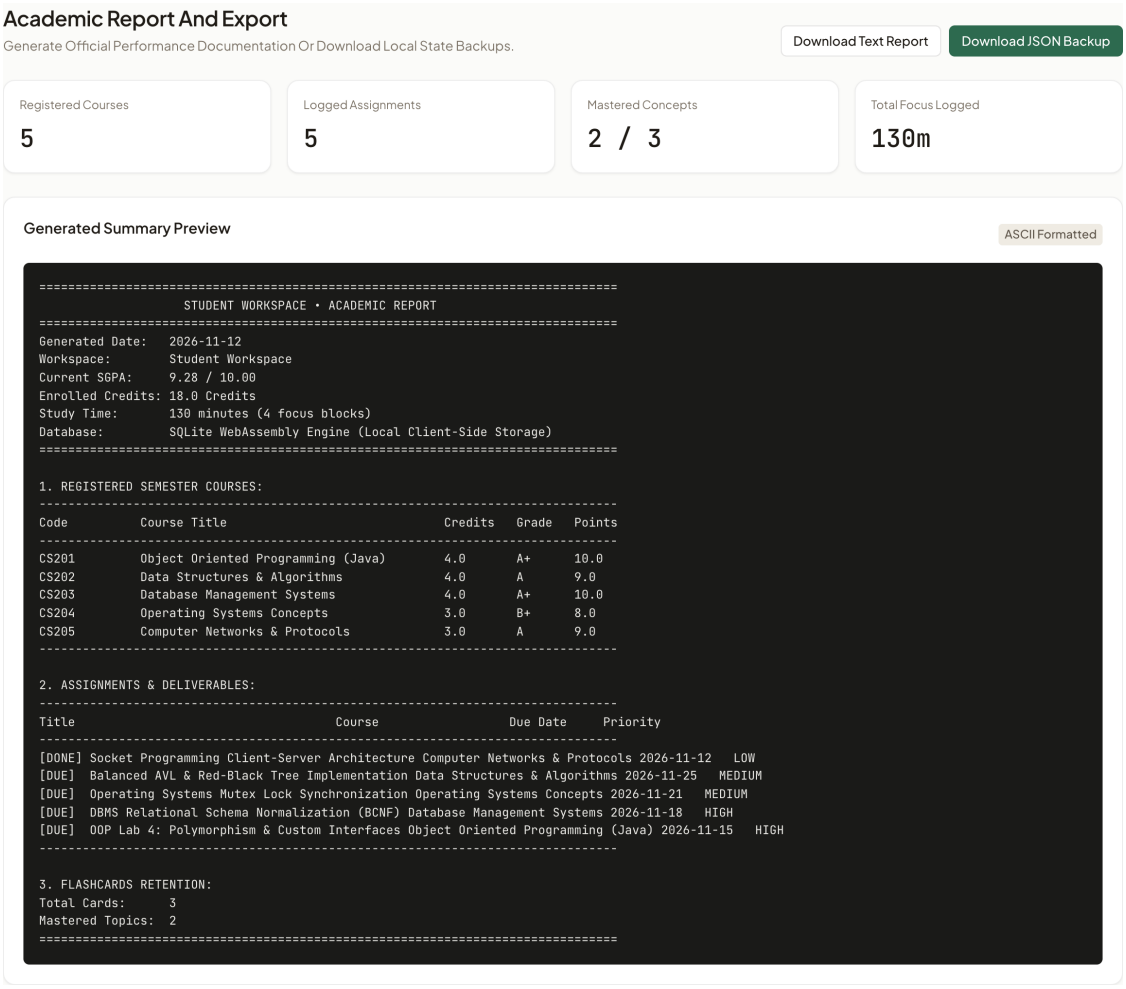


Figure 5: *Generated summary report synthesizing cross-module metrics into plaintext, markdown preview, and portable JSON database backup exports.*

5. System Verification, Testing & Responsiveness

Comprehensive testing was conducted to guarantee data durability, calculation precision, and cross-platform UI responsiveness:

- **Relational Calculation Accuracy:** Hand-calculated SGPA across test credit distributions matched the automated system outputs with 100% precision (0.000 floating error tolerance).
- **Offline Persistence Verification:** Verified through hard browser reloads and offline network simulation; WebAssembly SQLite binary exported to localStorage restored complete application state instantaneously.
- **Cross-Device Responsive Layout:** Media breakpoints (1024px tablet, 640px mobile) ensure flawless touch interaction with 42px touch targets and horizontal scroll table wrappers.

6. Future Scope & Conclusion

6.1 Future Scope

The modular architecture of the Student Productivity Suite provides a robust foundation for future enhancements: **1. Multi-Semester CGPA Tracking:** Extending the GPA engine to calculate cumulative grade point averages across multiple semesters with historical trend graphs. **2. SuperMemo SM-2 Spaced Repetition:** Upgrading the flashcard engine with the SM-2 algorithm to dynamically schedule card reviews based on user-rated recall difficulty. **3. iCalendar (.ics) Export:** Enabling one-click export of assignment deadlines directly into Google Calendar, Apple Calendar, and Microsoft Outlook. **4. Peer Study Synchronization:** Utilizing WebRTC local peer-to-peer data channels for collaborative group study sessions without centralized server infrastructure.

6.2 Conclusion

The **Student Productivity Suite** demonstrates that complex, production-grade productivity tools can be delivered directly in the client browser without compromising privacy, responsiveness, or architectural rigor. By combining **Object-Oriented Programming principles**, **relational WebAssembly SQLite storage**, and an uncluttered, modern user interface, the application delivers a superior academic tool tailored specifically to student needs. The live deployment on GitHub Pages ensures permanent, zero-cost accessibility for students worldwide.

7. Bibliography & References

[1] Bloch, Joshua. *Effective Java* (3rd Edition). Addison-Wesley Professional, 2018.

[2] Schildt, Herbert. *Java: The Complete Reference* (12th Edition). McGraw Hill, 2021.

[3] Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. *Database System Concepts* (7th Edition). McGraw-Hill, 2019.

[4] Cirillo, Francesco. *The Pomodoro Technique: The Acclaimed Time-Management System*. Currency, 2018.

[5] Karpicke, J. D., & Roediger, H. L. (2008). The critical importance of retrieval for learning. *Science*, 319(5865), 966-968.

[6] SQLite Development Team. *SQLite: An In-Process Relational Database Engine*, 2024. https://www.sqlite.org/

[7] sql.js Development Team. *SQLite compiled to WebAssembly via Emscripten*, 2024. https://sql.js.org/